

Data mining

Análisis descriptivo de las devoluciones en una tienda de moda

Pepa Morillas

y

Emilie Sellier

Indice de contenidos

- 1_Objetivos/definición del problema
 - 2_Selección de la base de datos
 - 3_Fase de limpieza & de transformación
 - 4_Análisis exploratorio
 - 5_Modelo de regresión logística
 - 6_Evaluacion del modelo y resultados
 - 7_Conclusiones
-

1_Objetivos/definición del problema



Definición del problema

En nuestra empresa de moda, queremos conocer qué productos se devuelven más & qué clientes devuelven más, características y motivos de las devoluciones.



Objetivos

Identificar & reducir los costes de stock de producto muchas veces obsoleto en tiendas físicas, incurrir en un ahorro en logística inversa & adaptar la oferta a nuestro clientes.



Cómo?

Analizando el por qué de las devoluciones, para anticiparnos y mejorar o eliminar determinados productos disponibles

2_ Selección de la base de datos

Estrategia 1

Combinar 3 bases de datos
⊕ mucha información detallada sobre los productos, precios...
⊖ difícil combinar la información con certezas

Estrategia 2

Tomar una base de datos específica sobre los devoluciones
⊕ calidad de datos más segura
⊖ espectro de información más estrecho

- Hemos elegido la **opción 2**, no necesitamos un espectro muy grande de datos pero sí una fiabilidad total.

Devolucoes (Devoluciones):

Devolucao_ID: Identificador único por producto

Produto_ID: Producto devuelto

Cliente_ID: Cliente que devuelve el producto

Quantidade: Cantidad devuelta

Motivo_Devolucao: Motivo de la devolución

Venda_ID: Identificador único por venta

Data da Devolução: fecha de devolución

- **Motivos de elección de estas bases de datos, justificación de uso**

- Los datos son estructurados, por lo que elegimos una base de datos relacional. Las bases de datos relacionales son escalables, lo que nos permite que el volumen de datos crezca significativamente a lo largo del tiempo. Hemos elegido una base de datos con una buena capacidad analítica.
- Motivo: Analizar eficientemente las razones de devolución de productos, para mejorar procesos, calidad y satisfacción del cliente.
- Justificación de uso: Se busca generar reportes periódicos o en tiempo real que apoyen áreas como logística, diseño y atención al cliente.

código de estrategia

1

```
import pandas as pd

productos = pd.read_csv("Produtos.csv")
devoluciones = pd.read_csv("Devolucoes.csv")
ventas = pd.read_csv("Historico_Vendas.csv")

devoluciones_clean = devoluciones.drop(['Quantidade', 'Venda_ID', 'Data da Devolução'], axis=1)
ventas_clean = ventas.drop(['Loja_ID', 'Colaborador_ID', 'Data da Venda', 'Cliente_ID',
                             'Venda_ID', 'Preço Unitário', 'Quantidade'], axis=1)
product_clean = productos.drop(['Imagem', 'Custo_Aquisição', 'Descrição', 'Nome'], axis=1)
total = pd.merge(product_clean, devoluciones_clean, on="Produto_ID")
total = pd.merge(total, ventas_clean, on="Produto_ID")

display(productos.head())
display(devoluciones.head())
display(ventas.head())
display(total.head())
```

```
import pandas as pd

devoluciones = pd.read_csv("Devolucoes.csv")
devoluciones_clean = devoluciones.drop(['Venda_ID', 'Data da Devolução'], axis=1)

display(devoluciones.head())
display(devoluciones_clean.head())
```

código de estratégia

2

	Devolução_ID	Venda_ID	Produto_ID	Cliente_ID	Quantidade	Motivo da Devolução	Data da Devolução
0	1	973	350	78	1	Expectativa não atendida	2024-01-25 03:14:57
1	2	787	207	213	2	Defeito no Produto	2016-12-08 11:49:28
2	3	153	25	51	4	Troca	2017-02-04 18:57:35

3_Fase de limpieza & de transformación

→ Columnas eliminadas :

Venda_ID: datos inútiles

Data da Devolução: los valores parecen erróneos,
horario nocturno o dominical

→ Realizamos la limpieza de datos, comprobando
valores nulos, omitidos y duplicados

→ Traducimos los datos de portugués a inglés

● Limpieza

Null values : 0

Missing values : 0

Duplicated lines : 0

Duplicated Return_ID : 0

→ Los datos están muy limpios; la ausencia de valores nulos nos indica, aunque no garantice que estén correctos, que los datos fueron bien recolectados, almacenados o procesados.

● Datos transformados

	Return_ID	Product_ID	Customer_ID	Quantity	Return reason
0	1	350	78	1	Unmet expectations
1	2	207	213	2	Defective product
2	3	25	51	4	Exchange

código de limpeza & transformação

```
# 1.Limpeza
print("Null values :")
print(devoluciones_clean.isnull().sum())
print("Missing values :")
print(devoluciones_clean.isna().sum())
duplicated = devoluciones_clean.duplicated()
print("Duplicated lines :",duplicated.sum())
duplicated = devoluciones_clean.duplicated(subset=['Return_ID'])
print("Duplicated Return_ID :",duplicated.sum())
```

```
# 2.Transformacion
return_reason = devoluciones_clean['Motivo da Devolução'].dropna().unique()
print("Return reason :")
for r in return_reason:
    print("-", r)
```

```
columns_translation = {
    'Devolução_ID' : 'Return_ID',
    'Produto_ID' : 'Product_ID',
    'Cliente_ID' : 'Customer_ID',
    'Quantidade' : 'Quantity',
    'Motivo da Devolução' : 'Return reason'
}
```

```
return_column_translation = {
    'Expectativa não atendida' : 'Unmet expectations',
    'Defeito no Produto' : 'Defective product',
    'Troca' : 'Exchange',
    'Tamanho Inadequado' : 'Inadequate size',
    'Outros' : 'Others',
    'Arrependimento' : 'Regret',
    'Desistência' : 'Drop out'
}
```

```
pdevoluciones_clean.rename(columns=columns_translation, inplace=True)
devoluciones_clean['Return reason'] = devoluciones_clean['Return reason'].replace(return_column_translation)
display(devoluciones_clean)
```

4_Análisis exploratorio

- Hemos observado las informaciones que podían ser útiles en una primera aproximación.
 - Nuestro objetivo es entender la estructura, la calidad y los patrones generales de los datos.
 - Revisamos la estructura de estos datos. Aunque ahora no necesitamos realmente un análisis exploratorio, en el futuro podemos cruzar estos datos con métricas de satisfacción o ventas.
 - Utilizamos fórmulas de estadística descriptiva y elegimos en este caso un diagrama de caja para visualizar los datos.
-

Forma general

Talla de la tabla (1000, 5)

Return_ID int64

Product_ID int64

Customer_ID int64

Quantity int64

Return reason object

dtype: object

→ Los tipos de datos están bien definidos en la base, es decir, cada columna del dataset tiene un tipo de dato correcto y coherente con el contenido que representa

Información general

Return reason :

Inadequate size 159

Unmet expectations 153

Exchange 142

Defective product 141

Regret 141

Drop out 139

Others 125

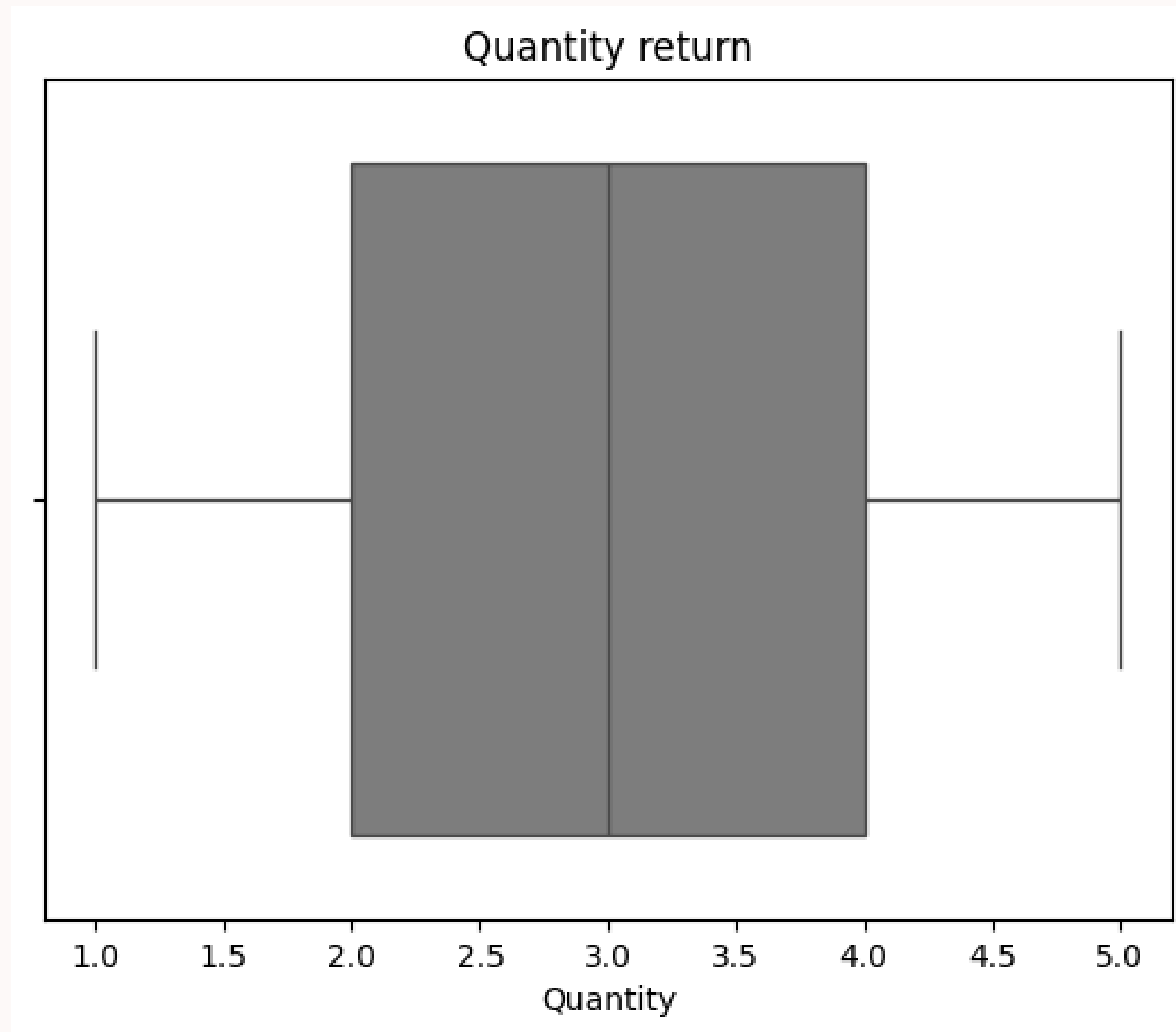
Product_ID: 636

Customer_ID: 640

→ No se han detectado incoherencias

Cantidad devuelta Box Plot

→ No hay valores extremos o fuera de cuadro para la variable Quantity, es decir, no se detectan outliers o valores atípicos según el criterio estadístico que usa este tipo de gráfico.

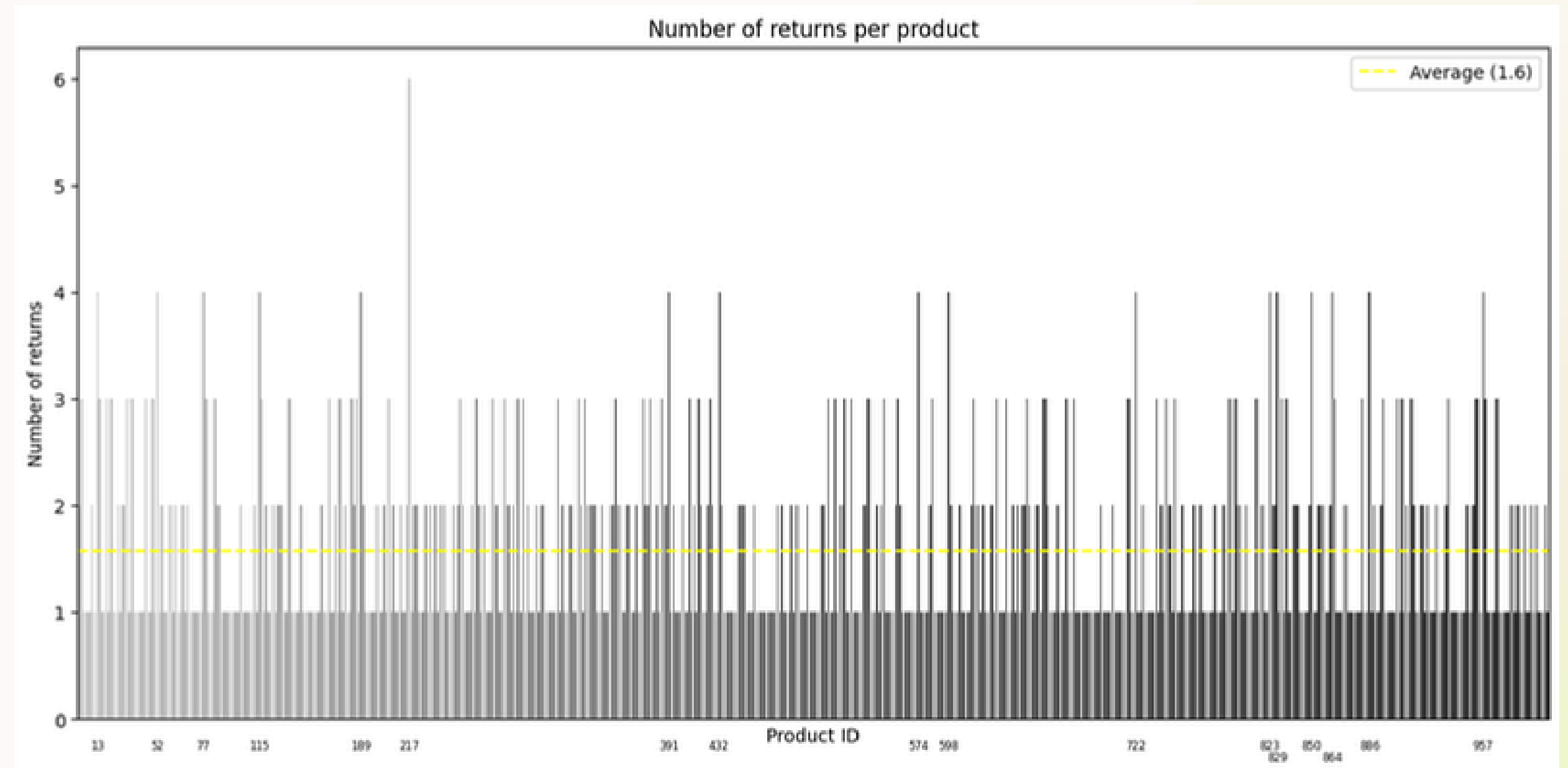


Product_ID valores medios y rendimientos fuera de campo

→ Average product returns:
1.5723270440251573

→ Standard deviation:
0.8090754478263065

→ Baja dispersión de las devoluciones, la mayoría esta entre 0,76 - 2,38

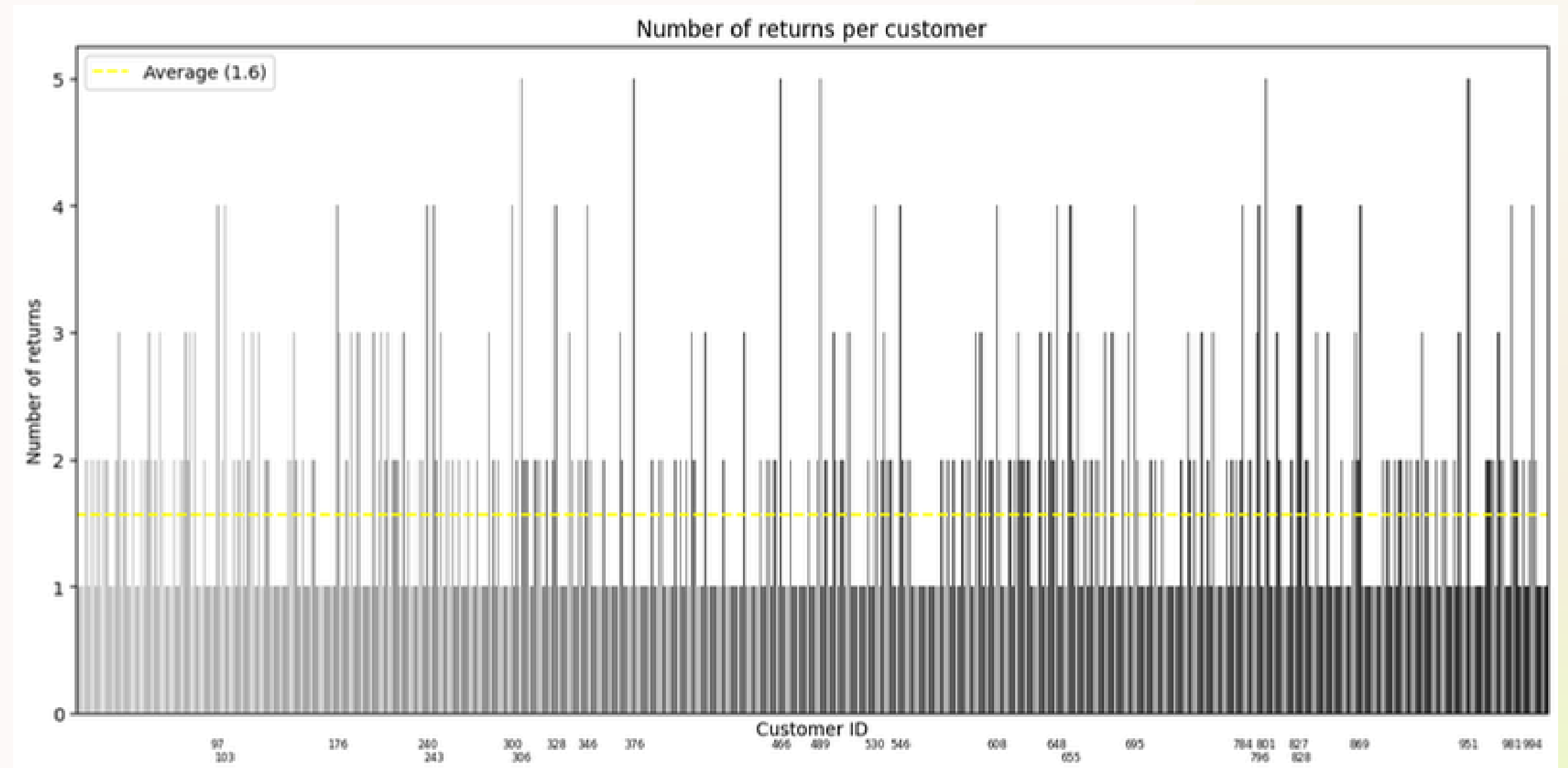


Customer_ID valores medios y rendimientos fuera de campo

→ Average customer returns:
1.5625

→ Standard deviation:
0.8424385202534875

→ Baja dispersión de las devoluciones, la mayoría esta entre 0,72 - 2,40



código de análisis exploratorio

```
print("Null values :")
print(devoluciones_clean.isnull().sum())
print("Missing values :")
print(devoluciones_clean.isna().sum())
duplicated = devoluciones_clean.duplicated()
print("Duplicated lines :",duplicated.sum())
duplicated = devoluciones_clean.duplicated(subset=['Return_ID'])
print("Duplicated Return_ID :",duplicated.sum())
```

```
#devoluciones_clean = devoluciones_clean.drop_duplicates() = no necessary because there is nothing to drop
display(devoluciones_clean)
```

```
import matplotlib.pyplot as plt
```

```
file = devoluciones_clean
```

```
#general shape
print(file.shape)
print(file.dtypes)
print(file.describe())
```

```
#general informations
print(file['Return reason'].value_counts())
print("Product_ID: ", file['Product_ID'].nunique())
print("customer_ID: ", file['Customer_ID'].nunique())
```



```
import seaborn as sns
```

```
sns.boxplot(data=file, x='Quantity', color='gray')  
plt.title("Quantity return")  
plt.show()
```

```
q1 = file['Quantity'].quantile(0.25)  
q3 = file['Quantity'].quantile(0.75)  
iqr = q3 - q1
```

```
lower_bound = q1 - 1.5 * iqr  
upper_bound = q3 + 1.5 * iqr
```

```
outliers = file[(file['Quantity'] < lower_bound) | (file['Quantity'] > upper_bound)]  
print(outliers)
```

código de Quantity Box Plot

código de exploration Product_ID

```
product_returns = file['Product_ID'].value_counts().sort_index()
mean_returns = product_returns.mean()
std_returns = product_returns.std()

# higher out_of_field return
high_return_products = product_returns[product_returns > mean_returns + 2 * std_returns]

print("Average product returns: ", mean_returns)
print("Standard deviation: ", std_returns)
print("Highly returned products: ", high_return_products)

plt.figure(figsize=(12, 6))
sns.barplot(x=product_returns.index.astype(str), y=product_returns.values, palette="Greys_d")
plt.axhline(mean_returns, color='yellow', linestyle='-', label=f'Average ({mean_returns:.1f})')

plt.title(f"Number of returns per product")
plt.xlabel("Product ID")
plt.ylabel("Number of returns")
plt.xticks([])
plt.legend()

## Display Product_ID with at least 4 returns
for i, (product_id, returns) in enumerate(product_returns.items()):
    if returns >= 4:
        if (product_id == 829 or product_id == 864):
            plt.text(i, -0.3, str(product_id), ha='center', va='top', fontsize=6, color='black')
        else:
            plt.text(i, -0.2, str(product_id), ha='center', va='top', fontsize=6, color='black')
plt.tight_layout()
plt.show()
```

código de exploration Customer_ID

```
customer_returns = file['Customer_ID'].value_counts().sort_index()
mean_returns = customer_returns.mean()
std_returns = customer_returns.std()

# higher out_of_field return
high_return_customers = customer_returns[customer_returns > mean_returns + 2 * std_returns]

print("Average customer returns: ", mean_returns)
print("Standard deviation: ", std_returns)
print("Highly returned customers: ", high_return_customers)

plt.figure(figsize=(12, 6))
sns.barplot(x=customer_returns.index.astype(str), y=customer_returns.values, palette="Greys_d")
plt.axhline(mean_returns, color='yellow', linestyle='-', label=f'Average ({mean_returns:.1f})')

plt.title(f"Number of returns per customer")
plt.xlabel("Customer ID")
plt.ylabel("Number of returns")
plt.xticks([])
plt.legend()

## Display Customer_ID with at least 4 returns
for i, (customer_id, returns) in enumerate(customer_returns.items()):
    if returns >= 4:
        if customer_id in [103, 243, 306, 655, 796, 828]:
            plt.text(i, -0.3, str(customer_id), ha='center', va='top', fontsize=6, color='black')
        else:
            plt.text(i, -0.2, str(customer_id), ha='center', va='top', fontsize=6, color='black')
plt.tight_layout()
plt.show()
```

5_Modelo de regresión logística

→ Regresión logística: Ideal para clasificación binaria (clientes/productos de riesgo).

→ Datos mixtos: Maneja variables categóricas (motivos) y continuas (n.º de devoluciones).

→ Fácil de interpretar: Rápido de implementar y útil para decisiones comerciales ágiles.

→ Probabilidades claras: Permite anticipar riesgos y optimizar stock y devoluciones.

→ Datos filtrados: Se usó una muestra relevante, no la base completa, para mayor precisión.

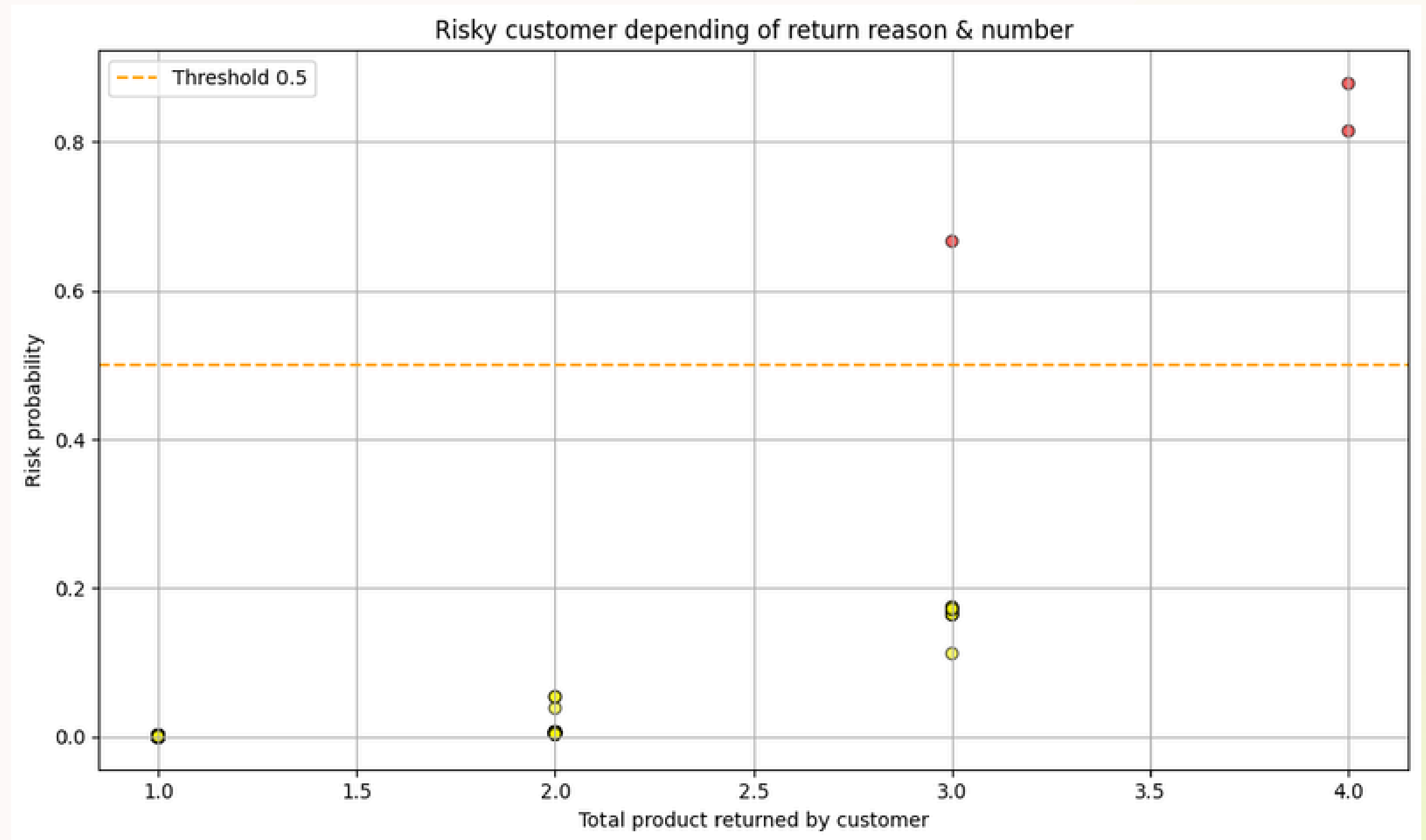
80% datos → para entrenar el modelo

20% datos → para probar y evaluar su rendimiento

Customer_ID, búsqueda de perfiles arriesgados

→ filtro con “**Unmet expectations**” & “**Regret**” y el número de devoluciones por cliente, a partir de 4, o de 3 con argumentos de retorno seleccionados = **en riesgo**

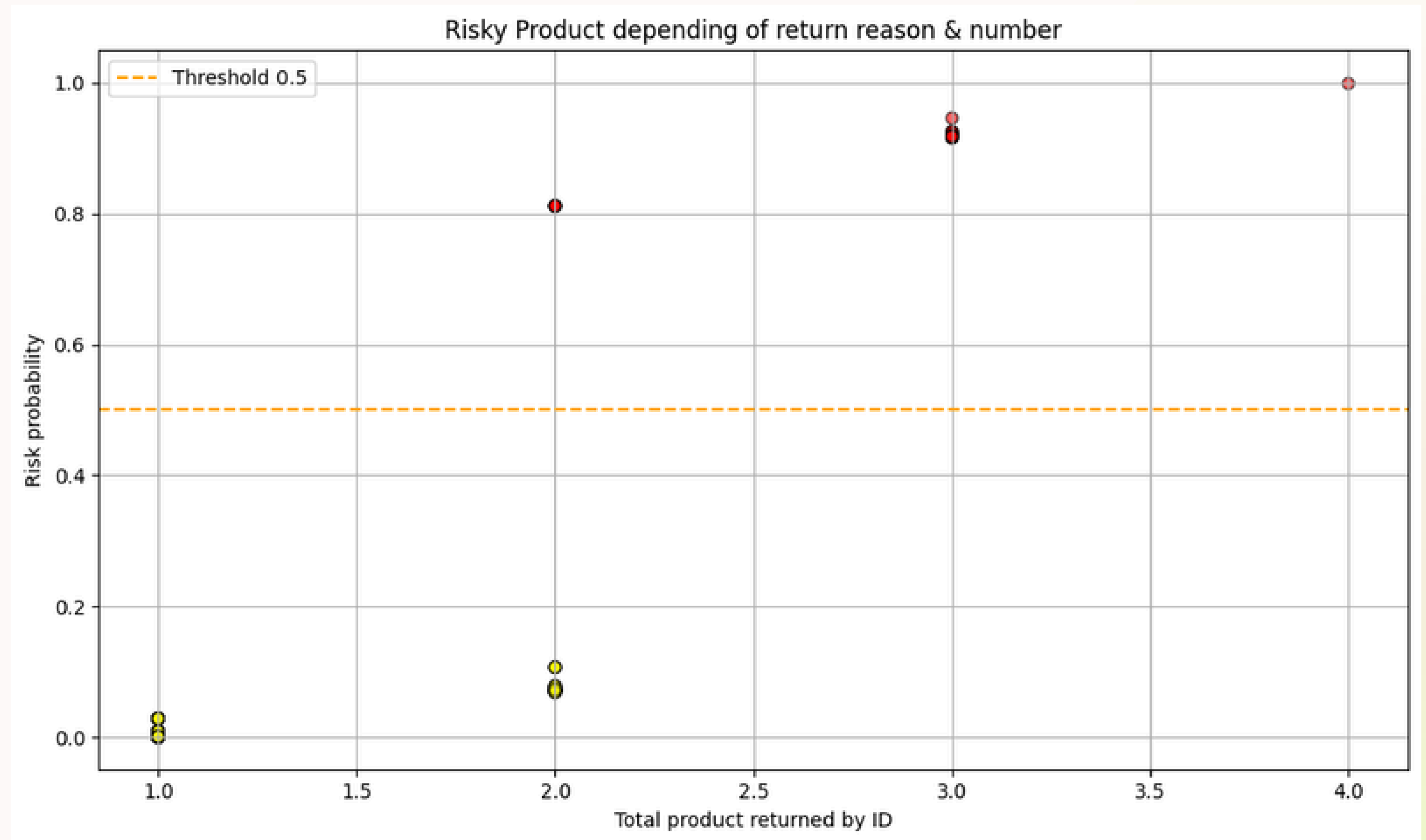
→ el umbral es de 0.5, arriba = un cliente en riesgo abajo = no en riesgo



Product_ID, búsqueda de referencias arriesgadas

-> filtro con “**Defective product**” & “**Inadequate size**” y el número de devoluciones por producto, a partir de 3 ,o de 2 con argumentos de retorno seleccionados = **en riesgo**

-> el umbral es de 0.5,
arriba = producto en riesgo
abajo = no en riesgo



```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score, classification_report
```

```
customer_data = customer_returns.rename('num_returns').reset_index()
customer_data.columns = ['Customer_ID', 'num_returns']
```

```
top_reasons = file.groupby(['Customer_ID', 'Return reason']).size().reset_index(name='count')
top_reason_per_customer = top_reasons.sort_values('count', ascending=False).drop_duplicates('Customer_ID')
```

```
customer_data = pd.merge(customer_data, top_reason_per_customer[['Customer_ID', 'Return reason']], on='Customer_ID')
encoder = OneHotEncoder(sparse_output=False)
reason_encoded = encoder.fit_transform(customer_data[['Return reason']])
reason_labels = encoder.get_feature_names_out(['Return reason'])
encoded_df = pd.DataFrame(reason_encoded, columns=reason_labels, index=customer_data.index)
```

```
X = pd.concat([customer_data[['num_returns']], encoded_df], axis=1)
y = ((customer_data['num_returns'] >= 4) |
     ((customer_data['num_returns'] == 3) &
      (customer_data['Return reason'].isin(['Unmet expectations', 'Regret'])))).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
y_probs = model.predict_proba(X_test)[:, 1]
y_pred = model.predict(X_test)
```

```
board = pd.DataFrame({
    'num_returns': X_test['num_returns'].values,
    'risk_proba': y_probs,
    'risky': y_test.values
})
plt.figure(figsize=(10, 6))
colors = ['yellow' if prob <= 0.5 else 'red' for prob in y_probs]
plt.scatter(board['num_returns'], board['risk_proba'], c=colors, alpha=0.6, edgecolor='k')
plt.xlabel("Total product returned by customer")
plt.ylabel("Risk probability")
plt.title("Risky customer depending of return reason & number")
plt.axhline(0.5, color='orange', linestyle='-', label="Threshold 0.5")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

código de regresión logística por cliente

código de regresión logística por producto

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score, classification_report
```

```
product_data = product_returns.rename('num_returns').reset_index()
product_data.columns = ['Product_ID', 'num_returns']
```

```
top_reasons = file.groupby(['Product_ID', 'Return reason']).size().reset_index(name='count')
top_reason_per_product = top_reasons.sort_values('count', ascending=False).drop_duplicates('Product_ID')
```

```
product_data = pd.merge(product_data, top_reason_per_product[['Product_ID', 'Return reason']], on='Product_ID')
encoder = OneHotEncoder(sparse_output=False)
reason_encoded = encoder.fit_transform(product_data[['Return reason']])
reason_labels = encoder.get_feature_names_out(['Return reason'])
encoded_df = pd.DataFrame(reason_encoded, columns=reason_labels, index=product_data.index)
```

```
X = pd.concat([product_data[['num_returns']], encoded_df], axis=1)
y = ((product_data['num_returns'] >= 3) |
     ((product_data['num_returns'] == 2) &
      (product_data['Return reason'].isin(['Defective product', 'Inadequate size'])))).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
y_probs = model.predict_proba(X_test)[:, 1]
y_pred = model.predict(X_test)
```

```
board = pd.DataFrame({
    'num_returns': X_test['num_returns'].values,
    'risk_proba': y_probs,
    'risky': y_test.values
})
plt.figure(figsize=(10, 6))
colors = ['yellow' if prob <= 0.5 else 'red' for prob in y_probs]
plt.scatter(board['num_returns'], board['risk_proba'], c=colors, alpha=0.6, edgecolor='k')
plt.xlabel("Total product returned by ID")
plt.ylabel("Risk probability")
plt.title("Risky Product depending of return reason & number")
plt.axhline(0.5, color='orange', linestyle='-', label="Threshold 0.5")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

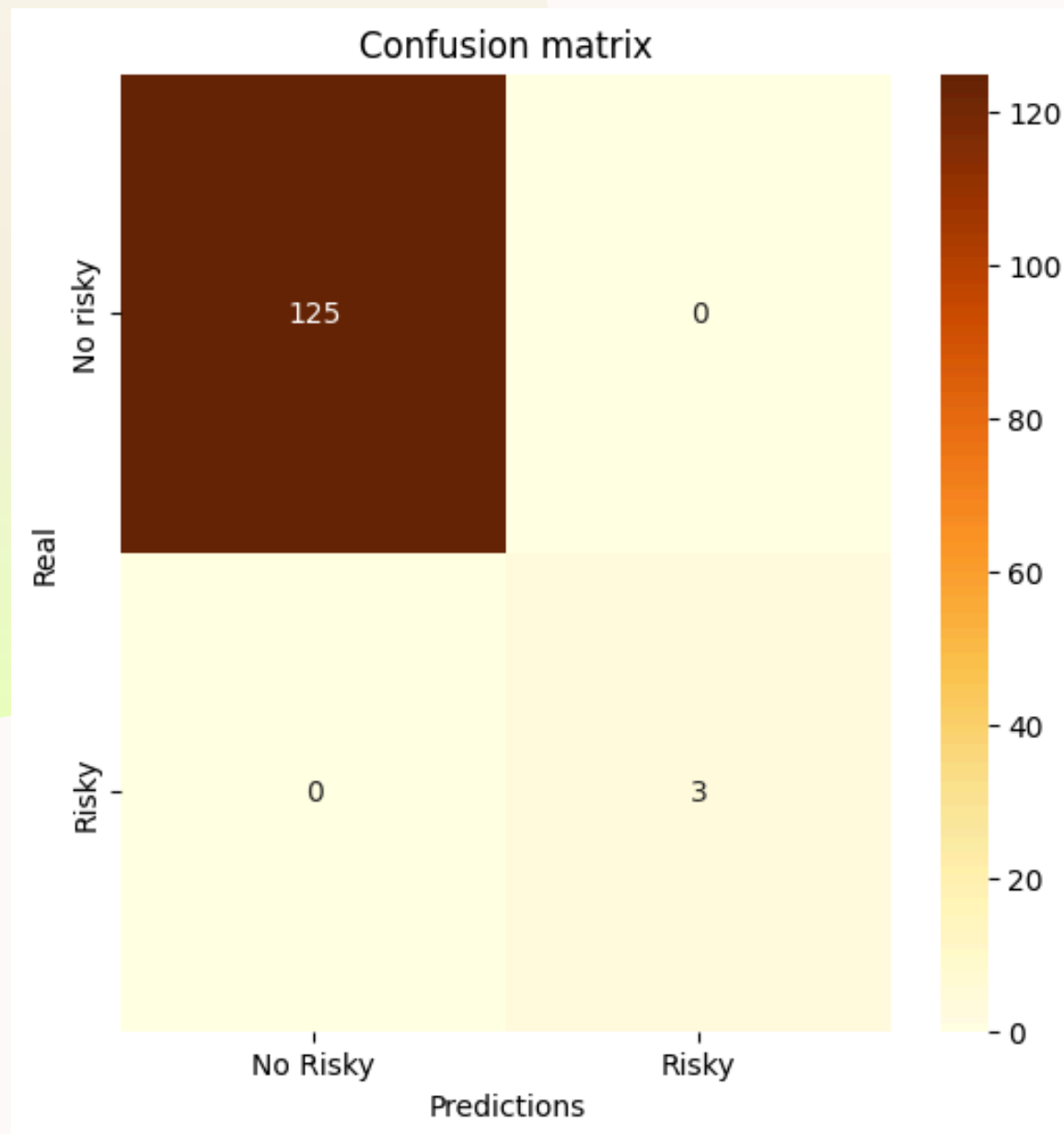

6_Evaluación del modelo y resultados

→ Para evaluar el rendimiento del modelo de clasificación, utilizamos varias métricas:

Matriz de confusión: muestra los verdaderos positivos, los verdaderos negativos, los falsos positivos y los falsos negativos. Esto ayuda a comprender los errores específicos del modelo.

Curva ROC: esta curva representa la tasa de verdaderos positivos (sensibilidad) frente a la tasa de falsos positivos para distintos umbrales.

Customer_ID, búsqueda de perfiles arriesgados



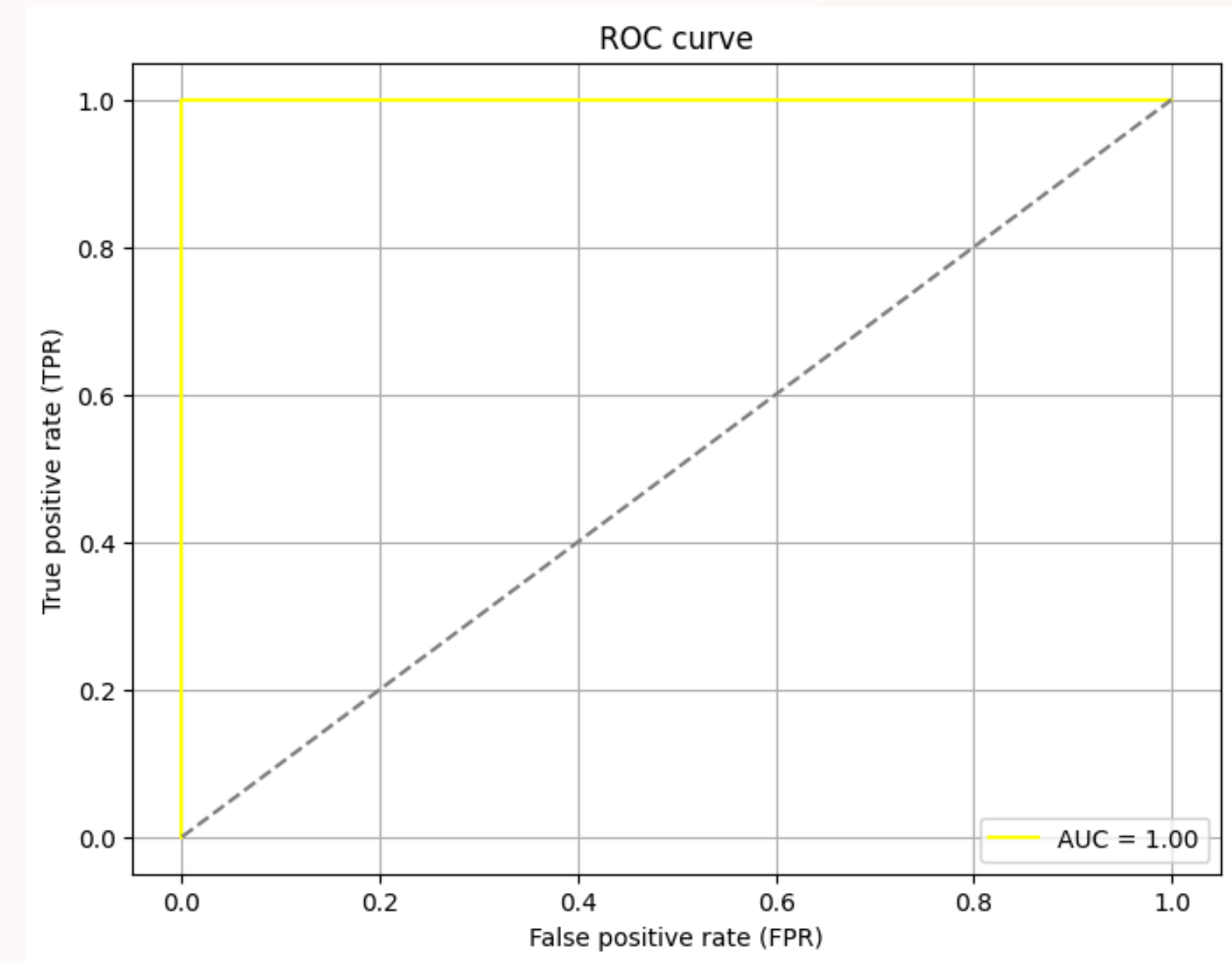
→ Para la matriz de confusión, el modelo consigue identificar a todos los clientes de riesgo

→ La curva muestra un ángulo agudo seguido de una línea próxima al máximo

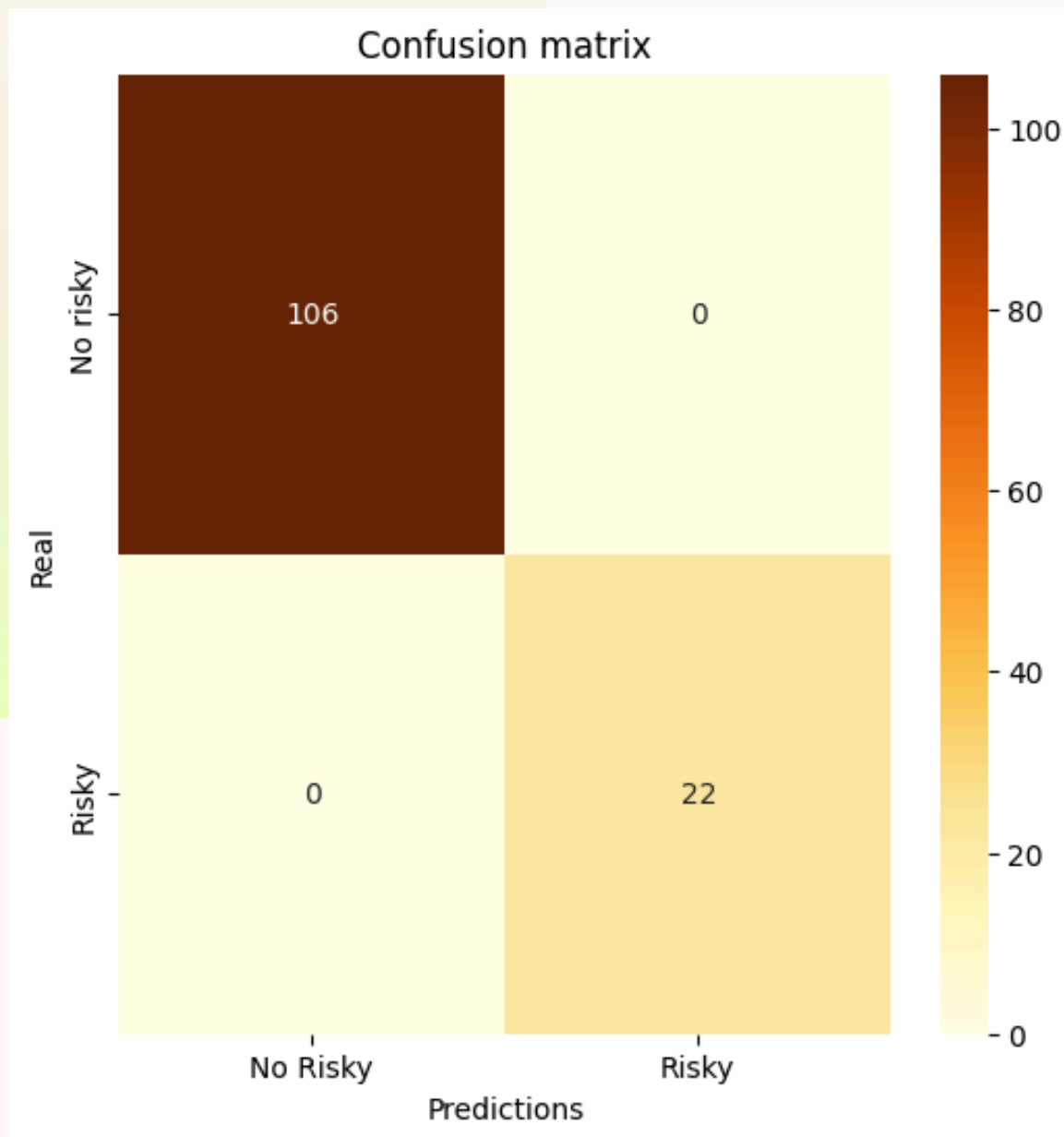
→ La calidad de la separación entre clases (AUC) resulta 1, lo que indica la excelente capacidad de distinción del modelo

→ Porcentaje de clientes de riesgo: 2,34%.

→ Clientes: Permite anticipar devoluciones y optimizar la logística de manera proactiva



Product_ID, búsqueda de referencias de riesgo



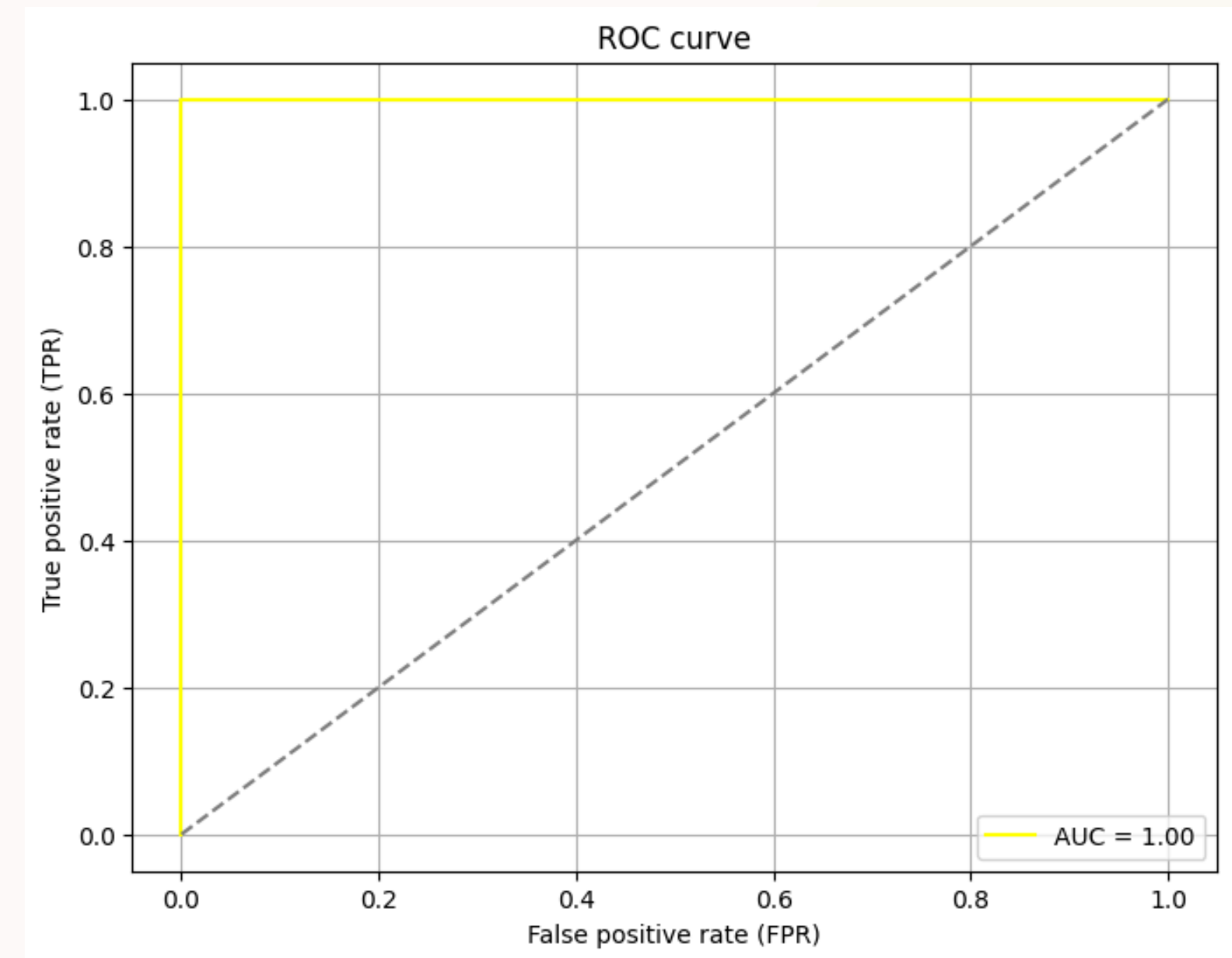
→ Para la matriz de confusión, el modelo consigue identificar todos los productos de riesgo

→ La curva muestra un ángulo agudo seguido de una línea próxima al máximo

→ La calidad de la separación entre clases (AUC) resulta 1, lo que indica la excelente capacidad de distinción del modelo

→ Porcentaje de productos de riesgo: 17.19%

→ Productos: Ayuda a entender las causas de las devoluciones y ajustar producción o marketing



código del evaluacion del modelo

```
#For customer
# 1. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='YlOrBr', xticklabels=['No Risky', 'Risky'], yticklabels=['No risky', 'Risky'])
plt.xlabel('Predictions')
plt.ylabel('Real')
plt.title('Confusion matrix')
plt.show()
```

```
# 2. Courbe ROC
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
auc = roc_auc_score(y_test, y_probs)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}', color = 'yellow')
plt.plot([0, 1], [0, 1], linestyle='-', color='gray')
plt.xlabel('False positive rate (FPR)')
plt.ylabel('True positive rate (TPR)')
plt.title('ROC curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
#For products
# 1. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='YlOrBr', xticklabels=['No Risky', 'Risky'], yticklabels=['No risky', 'Risky'])
plt.xlabel('Predictions')
plt.ylabel('Real')
plt.title('Confusion matrix')
plt.show()
```

```
# 2. Courbe ROC
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
auc = roc_auc_score(y_test, y_probs)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}', color = 'yellow')
plt.plot([0, 1], [0, 1], linestyle='-', color='gray')
plt.xlabel('False positive rate (FPR)')
plt.ylabel('True positive rate (TPR)')
plt.title('ROC curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

7_Conclusiones

→ Tras nuestros análisis exploratorios en profundidad y la aplicación de nuestro modelo de regresión logística, hemos recopilado algunos datos muy interesantes que pueden aplicarse a las decisiones estratégicas de la tienda.

• **Aplicación del modelo**

- **Modelo elegido:** La regresión logística fue clave por su simplicidad, eficacia y fácil interpretación
- **Anticipa devoluciones:** Identifica clientes propensos a devolver productos, según su historial y motivos frecuentes (como "expectativas no cumplidas" o "arrepentimiento")
- **Gestiona clientes de riesgo:** Permite ofrecer soluciones personalizadas y prevenir problemas logísticos
- **Optimiza el catálogo:** Detecta productos con alta probabilidad de devolución para ajustar estrategias de marketing, stock y comunicación



Gracias